MAIA

Monitoring Assistant & Intelligent Automation

Intelligent Monitoring & Auto-Healing Platform

System Specification Document

Version 2.2 · July 2026

Table of Contents

1 Executive Summary
2 System Architecture
3 Authentication & Authorization
4 Key Stakeholders & User Roles
5 Intelligence Model & Future Direction
6 Operator Workflow
7 Auto-Heal Workflow
8 Configuration Management
9 Feature Status
10 Glossary
A Appendix — Clean Architecture Layers
B Appendix — Background Worker Pipeline
C Appendix — End-to-End Data Flow
D Appendix — Fix Execution Engine
E Appendix — Scan Strategies
F Appendix — Database Schema
G Appendix — API Endpoint Catalog
H Appendix — Non-Functional Requirements
I Appendix — Known Follow-ups

1 Executive Summary

MAIA — **Monitoring Assistant & Intelligent Automation** — is a rule-based intelligent automation platform for data pipeline operations. It monitors any configured job type — DTSX packages, executables, Windows services, or any process whose errors can be scanned from log files, database tables, API endpoints, or structured file content (XML).

MAIA detects failures, classifies them using configurable pattern-matching rules, generates fix recommendations from a policy catalogue (including multi-step composite fixes), and either resolves them automatically via auto-heal or routes them to an operator through a modern Angular dashboard with slide-in failure drawers, keyboard navigation, and dark mode support.

The system ships with full authentication (PBKDF2 sessions, three-tier roles — User, Operator, Administrator), immutable audit logging with a dedicated admin UI, and is designed for offline, air-gapped deployment. Every decision is deterministic and traceable.

Faster Detection	Less Manual Work	Full Audit Trail	Self-Service Config
4 scan strategies detect failures in seconds	Auto-heal + composite fixes eliminate repetitive work	Every action logged immutably with EntityType/Id	Full CRUD via web UI — role-gated, no DB access

2 System Architecture

MAIA follows Clean Architecture with a Tier 2.5 ScanSource entity model: each MonitoredJob contains N typed ScanSources, each with its own rules and watermarks.

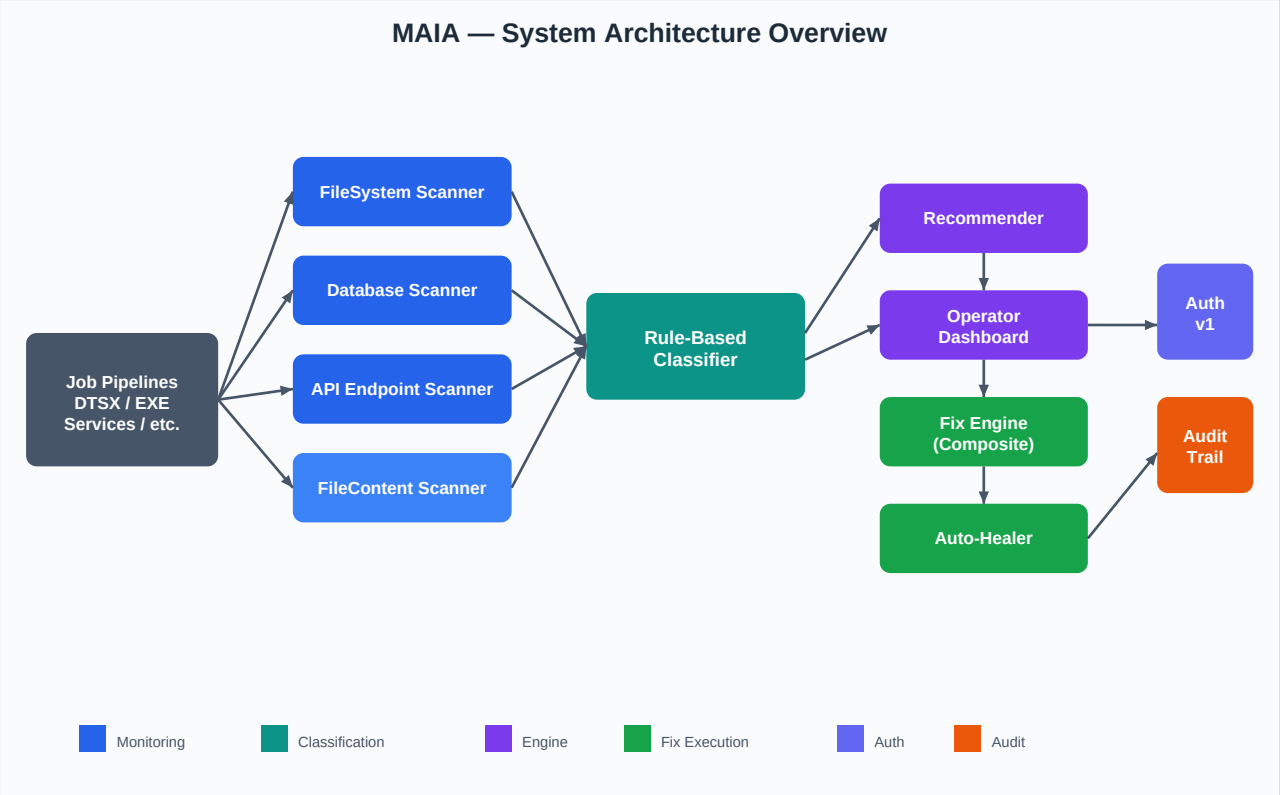


Figure 1 — System Architecture Overview

Technology Stack

Concern	Technology
Backend	C# / .NET 8 / ASP.NET Core / Entity Framework Core / Serilog

Frontend	Angular 20.2, TypeScript 5.9, RxJS 7.8, chart.js@4 — standalone components, lazy routes, dark mode
Database	SQL Server (EF Core migrations, 21 tables)
Auth	PBKDF2-HMAC-SHA512, httpOnly cookie sessions, SameSite=Strict, 3-tier roles
Background	MonitoringWorker (lease-coordinated) + ScanHistoryRetentionWorker
Deployment	IIS (prod) or containerised. Multi-instance safe via MonitoredJobLeases.
Testing	xUnit (backend, 275+ tests incl. auth matrix), Karma/Jasmine (frontend)

3 Authentication & Authorization

MAIA ships with local username/password authentication, server-side sessions, and three-tier role enforcement. Every API endpoint is gated by default (fallback = RequireAuthenticatedUser).

Role	Permissions
User	Read-only access. Dashboard, failures list, scan history. Cannot approve/reject or configure.
Operator	All User permissions + approve/reject recommendations, retry fixes, mark resolved, trigger scans.
Administrator	All Operator permissions + configuration CRUD (jobs, rules, policies), user management, audit log viewer, worker pause/resume.

Security controls:

Control	Detail
Password hashing	PBKDF2-HMAC-SHA512, 100k iterations, per-password random salt
Sessions	httpOnly cookie, SameSite=Strict, configurable idle timeout (default 1h), throttled sliding
Forced rotation	MustChangePasswordMiddleware blocks all API calls until seed password is changed. Non-skippable in production.
Seed credential	admin/admin — usable only for its own rotation. Forced rotation is the mitigation.
Actor tracking	All audit + operator action records use server-derived username from cookie principal

4 Key Stakeholders & User Roles

Role	Who	What They Do
User (Tier 1)	Support / Junior Engineer	Views dashboard, failures, scan history. Read-only.
Operator (Tier 2)	Data / Support Engineer	Reviews failures, approves/rejects/retries fixes, marks resolved, sets auto-heal, triggers scans.
Administrator (Tier 3)	Senior Engineer / Lead	Configures jobs, sources, rules, policies. Manages users. Views audit log. Controls worker.
System (Auto)	MAIA itself	Runs scans, classifies, generates recommendations, executes auto-heal and composite fixes.
Auditor	Compliance / Management	Reviews the immutable audit log (global /config/audit screen).

5 Intelligence Model & Future Direction

Current: deterministic rule-based automation

No ML or LLM. Every decision traces to a configuration record. Classification uses wildcard pattern matching (* = any text). Fix dispatch uses (JobTypeId, ErrorTypeId) policy lookup with optional per-MonitoredJob overrides. Composite policies run ordered steps best-effort.

Path to AI-augmented monitoring (next generation)

Capability	Description
LLM classification	Classify novel errors that don't match rules. RuleBasedClassifier runs first; unmatched fall through to LLM.
Dynamic fix suggestions	LLM generates candidate fixes for unconfigured error types. Operator approval still required.
Anomaly detection	ML-based detection beyond threshold rules (row count drift, latency trends).
Natural language config	Operators describe intent in plain language; system generates draft rules for admin review.

Extension Points Already in Place

- IClassificationStrategy — swap or chain classifiers
- IFixCatalogue — add LLM-backed catalogue alongside DbFixCatalogue
- IScanStrategy — register new strategies without modifying existing ones
- Command palette built forward-compatible with planned LLM assistant (Track 1 in AI_INTEGRATION.md)

6 Operator Workflow

Operators review failures through a slide-in drawer (760px, keyboard-navigable). Clicking any failure row opens the drawer with full context, recommendations, execution history, and action buttons.

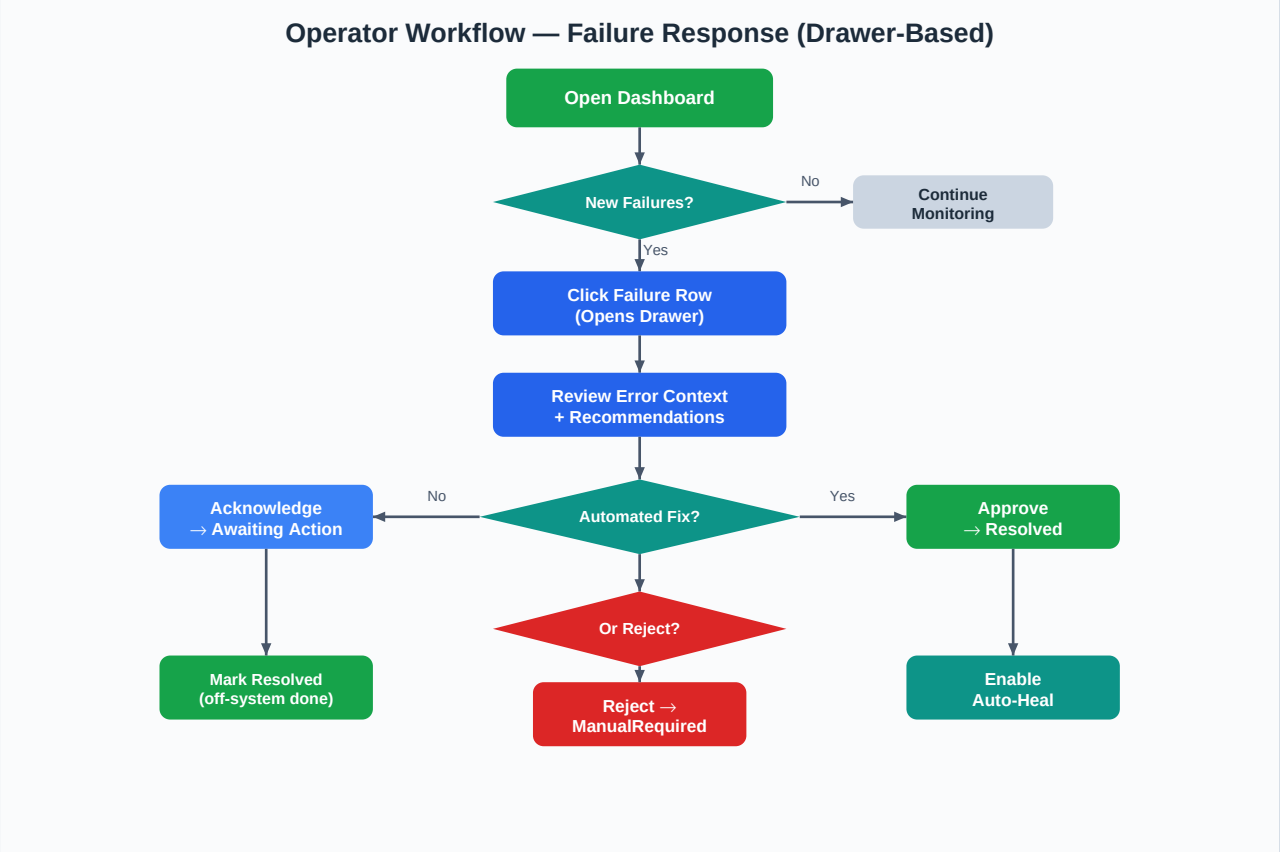


Figure 2 — Operator Failure Response Workflow

Drawer features:

Feature	Description
Keyboard navigation	↑/↓ between rows, Enter opens drawer, Esc closes. Auto-loads next/prev page at boundaries.
Smart back button	← Back to {Dashboard/Recommendations/...} based on navigation history.
Live polling	5s silent refresh of failure status. Buttons disable gracefully if state changes mid-review.
Execution history	Collapsible per-attempt list with ✓/✗ per step. Composite steps shown inline.
Mark Resolved	Available on AwaitingManualAction and ManualRequired. Operator's exit ramp for off-system fixes.
Retry Fix	Re-arms a failed execution with current policy. Explicit operator override.

7 Auto-Heal Workflow

Auto-heal resolves known failures without operator intervention. The MonitoringWorker atomically claims both jobs (via leases) and recommendations (via atomic UPDATE), executes fixes, and records results.

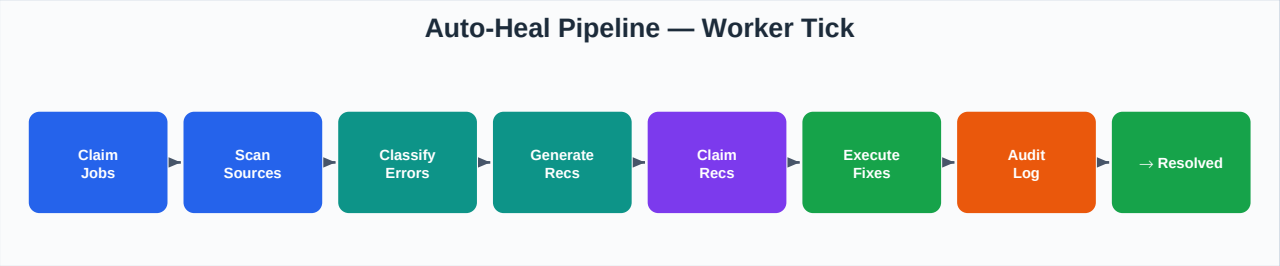


Figure 3 — Auto-Heal Pipeline

Auto-Heal Controls

- FixPolicyRule.IsAutoHealEligible = true for the (JobType, ErrorType) pair
- Toggle on Recommendations screen mutates the policy (affects future recs)
- Read-only Auto-run badge shows the frozen snapshot on the individual recommendation
- Recommendation atomic claim prevents double-execution across concurrent drains

8 Configuration Management

Admins configure MAIA through the web UI. The Tier 2.5 model separates MonitoredJobs (identity) from ScanSources (typed observation points with their own rules).

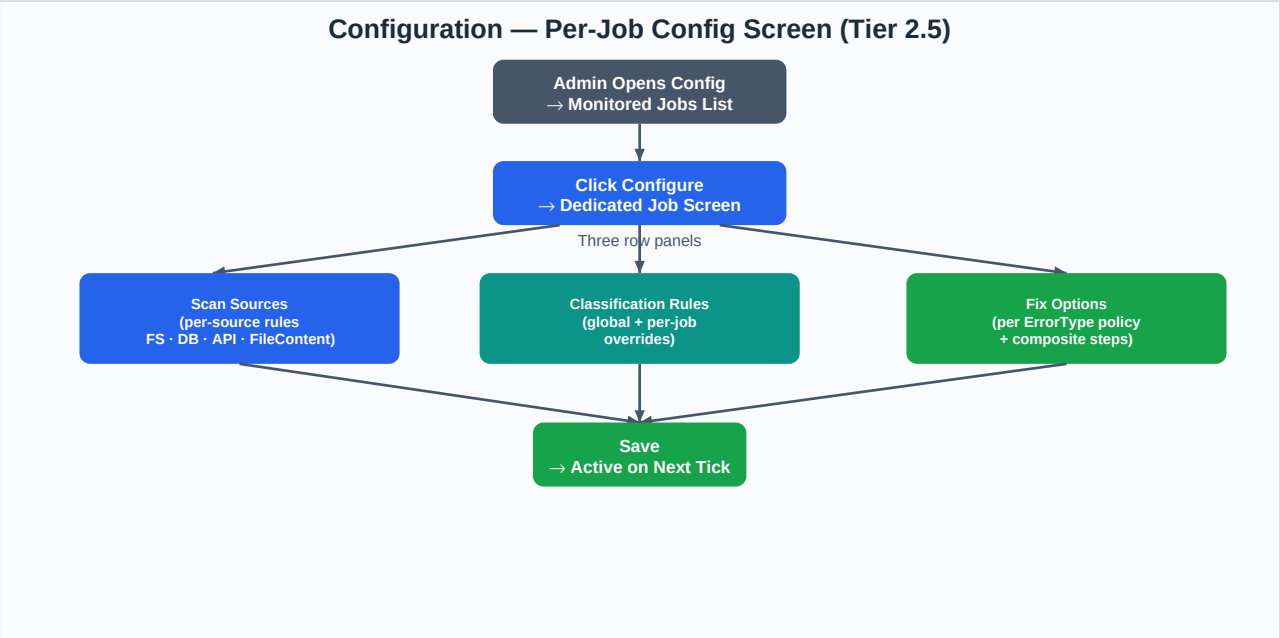


Figure 4 — Per-Job Configuration Screen

Job Flow view:

A read-only comprehension surface traces each scan rule's detect → classify → recommend → fix path. Shows coverage gaps, auto-heal vs manual routing, and classification-with-no-fix states. Operator+ only.

9 Feature Status

All features below are built and operational as of v2.2 (July 2026).

Monitoring

FileSystem scan — ErrorKeyword, wildcard stripping, byte-offset watermarks, per-line failure emission	Built
Database scan — ColumnRange + ValueEquals rules, filtered watermark advancement	Built
API Endpoint scan strategy	Built
FileContent scan — XML extractor, mtime-based ScanContentWatermarks	Built
Tier 2.5 ScanSource entity — per-job sources with nested rules, dedicated config screen	Built
Per-ScanType lease duration + per-tick ScanRunHistory + 30-day retention worker	Built

Classification

RuleBasedClassifier — wildcard (*) patterns, priority-ordered, per-job overrides	Built
(JobTypeId, ErrorTypeId) dual-key policy lookup everywhere	Built
ErrorType CRUD (soft-delete) via /api/config/error-types	Built

Fix Execution

5 executors: ApiCall, StoredProcedure, SqlScript, CopyFile, Manual — all with per-step timeout	Built
Composite fix policies — ordered FixPolicyRuleSteps, best-effort execution, per-step logging	Built
Recommendation atomic claim — READPAST+UPDLOCK, ClaimedBy/ClaimedAt, closes double-exec race	Built
FixResult/FixOutcome — Success/NoAutomatedAction/Failed with detail strings	Built
Detect-then-drain + centralised drain (startup + post-tick + approve + manual + classify-pending)	Built
IPlaceholderResolver — {failureId}, {sourceId}, {sourceFilePath}, {sourceFileName}, {jobFolder}, {inputFolder}	Built

Operator UI

Dashboard — 6 KPI tiles (Active, Awaiting, ManualReq, FixFailed, Unconfigured, Resolved), 2 charts	Built
Failures drawer — 760px slide-in, keyboard nav (↑↓/Enter/Esc), live 5s polling, execution history	Built
Recommendations drawer + Operator Actions history (split views)	Built
Job Flow view — per-job detect→classify→recommend→fix path visualization	Built
Per-job config screen — 3 row panels (Sources, Classification, Fix Options), drawer editors	Built
Audit Log viewer — global /config/audit, admin-only, paged/filtered	Built
Dark theme — OS default + manual toggle, full token coverage	Built
Command palette — Ctrl/Cmd+K, nav + failure-by-id + job search, role-filtered	Built
Server-side sortable failures list + URL-as-source-of-truth filters	Built
Data staleness indicator — top-bar amber chip when worker heartbeat fails	Built

Authentication & Infrastructure

Auth v1 — PBKDF2 sessions, 3-tier roles, forced password rotation, auth matrix tests	Built
Serilog file logging — daily rolling + 30-day retention, config-only verbosity	Built
ConfigController full audit coverage — all 5 entities, EntityType/EntityId shape	Built
Lease-coordinated MonitoringWorker + ScanHistoryRetentionWorker	Built
GlobalExceptionHandler — RFC 7807 ProblemDetails	Built
Language switcher scaffold — English active, Hebrew 'soon'	Built

Planned — Next Generation

LLM-assisted classification for novel/unmatched error patterns	Planned
Dynamic fix suggestions via language model	Planned
ML-based anomaly detection	Planned
Full RTL/Hebrew translations + layout mirroring	Planned
Dashboard Phase 2 — Failures by Job + Resolution Mix charts	Planned

10 Glossary

Term	Definition
MonitoredJob	A configured pipeline/process. Can be DTSX, EXE, Service, etc. Contains N ScanSources.
ScanSource	A typed observation point within a job (Tier 2.5). Has its own ScanType, config fields, and rules.
ScanCheckRule	A condition evaluated during a scan: ColumnRange, ValueEquals, ErrorKeyword, FileContent.
ClassificationRule	A wildcard pattern that assigns JobType + ErrorType to a failure. Priority-ordered; first match wins.
JobFailure	A detected failure instance with SourceId, StepName, ErrorMessage, Status, SourceFilePath.
Recommendation	A fix suggestion from FixPolicyRule lookup. Carries ConfidenceScore, AutoFixAvailable, ClaimedBy.
FixPolicyRule	Keyed by (JobTypeId, ErrorTypeId) + optional MonitoredJobId override. ActionType + Steps for composite.
Auto-Heal	Fix execution without operator approval, governed by IsAutoHealEligible. Every auto-heal is audited.
Composite Fix	A FixPolicyRule with ActionType=Composite and ordered FixPolicyRuleSteps. Steps run best-effort.
AwaitingManualAction	Status for operator-acknowledged manual fixes. Exit via Mark Resolved.
Detect-then-Drain	Scan strategies detect/classify/suggest only. Fixes are centralised in the worker drain.
ScanSource Watermark	Position tracking per source type: byte offset (FS), ISO datetime2 (DB), mtime (FileContent).
AuditLog	Immutable trail with EntityType + EntityId. Covers fix executions, operator actions, and config changes.

TECHNICAL APPENDIX

A Appendix — Clean Architecture Layers

Layer	Components	Responsibility
Presentation (API)	DataController, ConfigController, RecommendationsController, FailuresController, JobScanController, AdminController, AuthController, UsersController + GlobalExceptionHandler	HTTP endpoints, RFC 7807 errors, role enforcement
Application (Use Cases)	ClassifyJobsUseCase, GenerateSuggestionsUseCase, ExecuteFixesUseCase, DirectoryPipelineUseCase, ScanHistoryRetentionService, AuthService	Orchestrate domain logic
Core (Domain)	Entities (21 tables), Interfaces (12 repos + strategies + engine), Enums (JobStatus incl. AwaitingManualAction, FixOutcome, FixActionType incl. Composite/CopyFile)	Business rules & contracts
Infrastructure	SqlRepositories (12×), Scan Strategies (4×), Fix Executors (6×), Fix Handlers (4×), RuleBasedClassifier, DbFixCatalogue, PlaceholderResolver, 2× BackgroundServices, Auth handler	Implements Core interfaces

B Appendix — Background Worker Pipeline

MAIA runs **two** background workers.

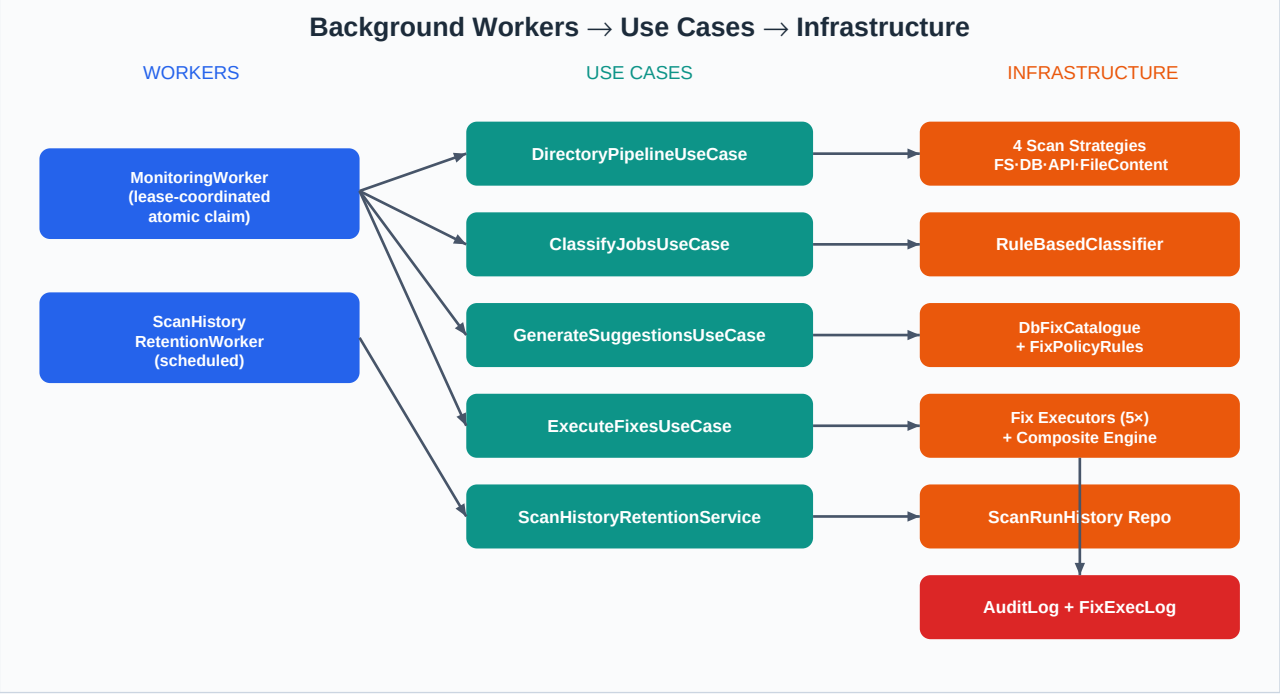


Figure B1 — Worker Pipeline

Worker	Behaviour

MonitoringWorker	Lease-coordinated. Startup drain → loop: claim up to 4 jobs atomically → parallel scan (degree=4, per-ScanType timeout) → post-tick drain of atomically-claimed recommendations → release leases + write ScanRunHistory. Pausable via admin endpoint.
ScanHistoryRetention Worker	Scheduler. Startup + every CleanupIntervalHours (default 6). Deletes ScanRunHistory older than RetentionDays (default 30). Config hot-reloaded.

C Appendix — End-to-End Data Flow

Stage	Description
1 — Detect	IScanStrategy.ScanAsync reads new data from each ScanSource (4 types). Applies ScanCheckRules. Saves JobFailure records with SourceFilePath.
2 — Classify	ClassifyJobsUseCase: RuleBasedClassifier applies wildcard patterns in Priority order. First match assigns JobTypeId + ErrorTypeId.
3 — Suggest	GenerateSuggestionsUseCase: FixPolicyRule lookup by (JobTypeId, ErrorTypeId) + optional MonitoredJobId override. Idempotent (skips existing recs).
4a — Auto-Heal	Worker post-tick drain: atomically claims pending recs, dispatches to executor (single or composite), writes FixExecutionLog + AuditLog.
4b — Operator	Approve (synchronous drain), Reject (last-pending → ManualRequired), Acknowledge (Manual → AwaitingManualAction), Mark Resolved, Retry Fix.
5 — No policy	DefaultFixEngine logs 'Policy lookup miss', falls to FixCategory handler. DbFixHandler is a stub → ManualRequired.
6 — Unclassified	ErrorTypeId stays NULL. No recommendation generated. Operator/admin must add a ClassificationRule.

D Appendix — Fix Execution Engine

ExecuteFixesUseCase atomically claims pending recommendations, then dispatches to the appropriate executor. Composite policies run ordered steps best-effort.

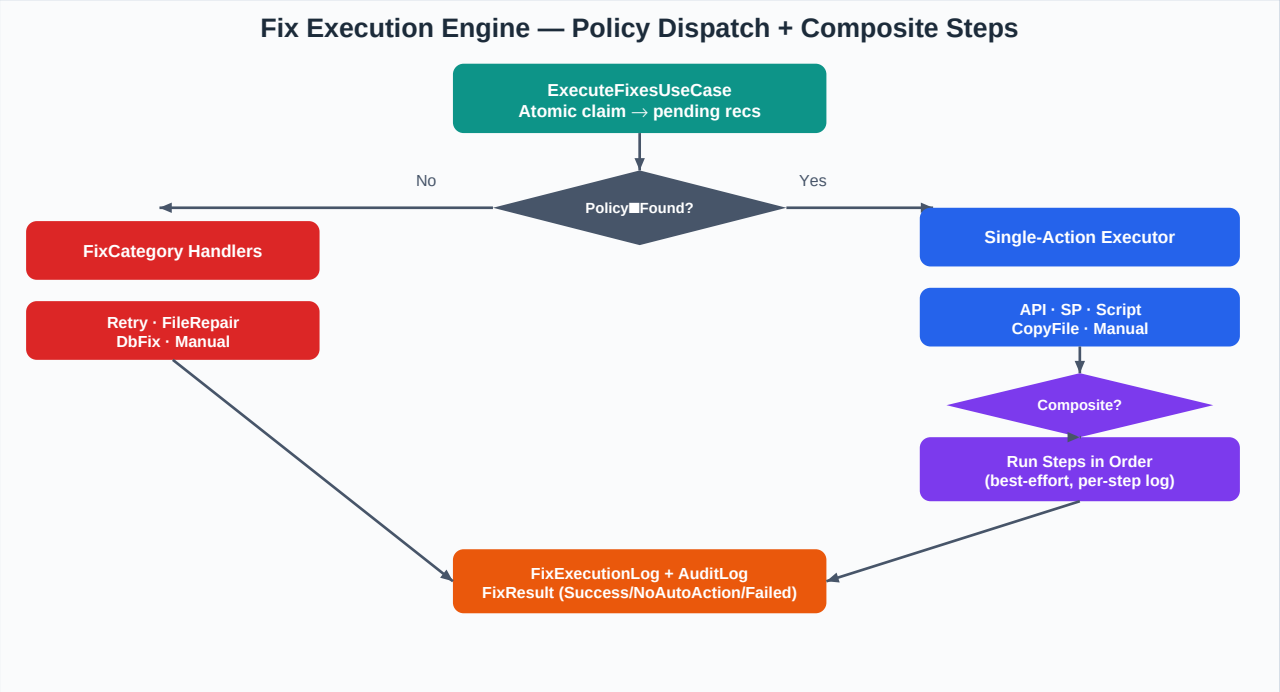


Figure D1 — Fix Execution Engine

Executor	ActionType	Behaviour
ApiCallExecutor	ApiCall	HTTP POST to URL with placeholders. Per-request CTS timeout (60s).
StoredProcedureExecutor	StoredProcedure	EXEC against MAIA DB connection. CommandTimeout honoured.
SqlScriptExecutor	Script	SQL against target MonitoredJob.ConnectionName. {failureId}/{sourceId}/{sourceFilePath} placeholders. 120s timeout.
CopyFileExecutor	CopyFile	Atomic copy via .tmp + rename. UNC supported. Overwrite default. Async stream-based (81KB chunks).
ManualActionExecutor	Manual	Returns NoAutomatedAction. Routes to AwaitingManualAction (operator-approved) or ManualRequired (auto-path).

E Appendix — Scan Strategies

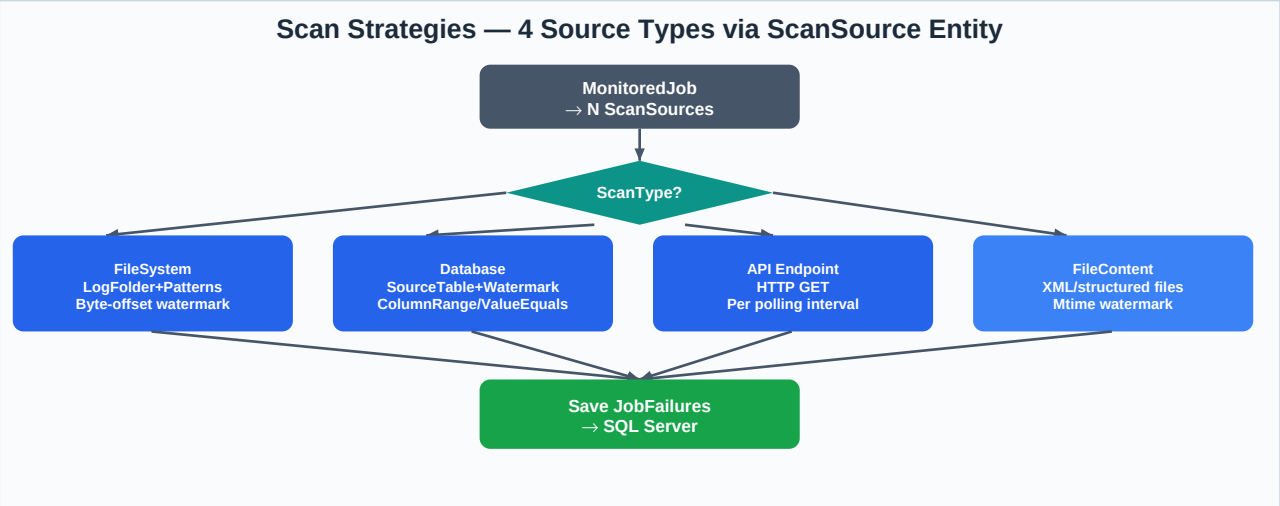


Figure E1 — Scan Strategy Routing via ScanSource

Strategy	Input	Watermark	Check Rules
FileSystemScanStrategy	Log files in source LogFolder	ScanFileWatermark (byte offset)	ErrorKeyword + InputPathPattern
DatabaseScanStrategy	SourceTable rows > watermark	ScanDbWatermark (ISO datetime2)	ColumnRange, ValueEquals, FilePathColumn
ApiEndpointScanStrategy	HTTP GET from LogSourceUrl	Per polling interval	Response body signal
FileContentScanStrategy	XML files via XmlContentExtractor	ScanContentWatermark (mtime)	FileContent check rules

F Appendix — Database Schema

MAIA uses SQL Server with 21 tables. AuditLog is append-only.

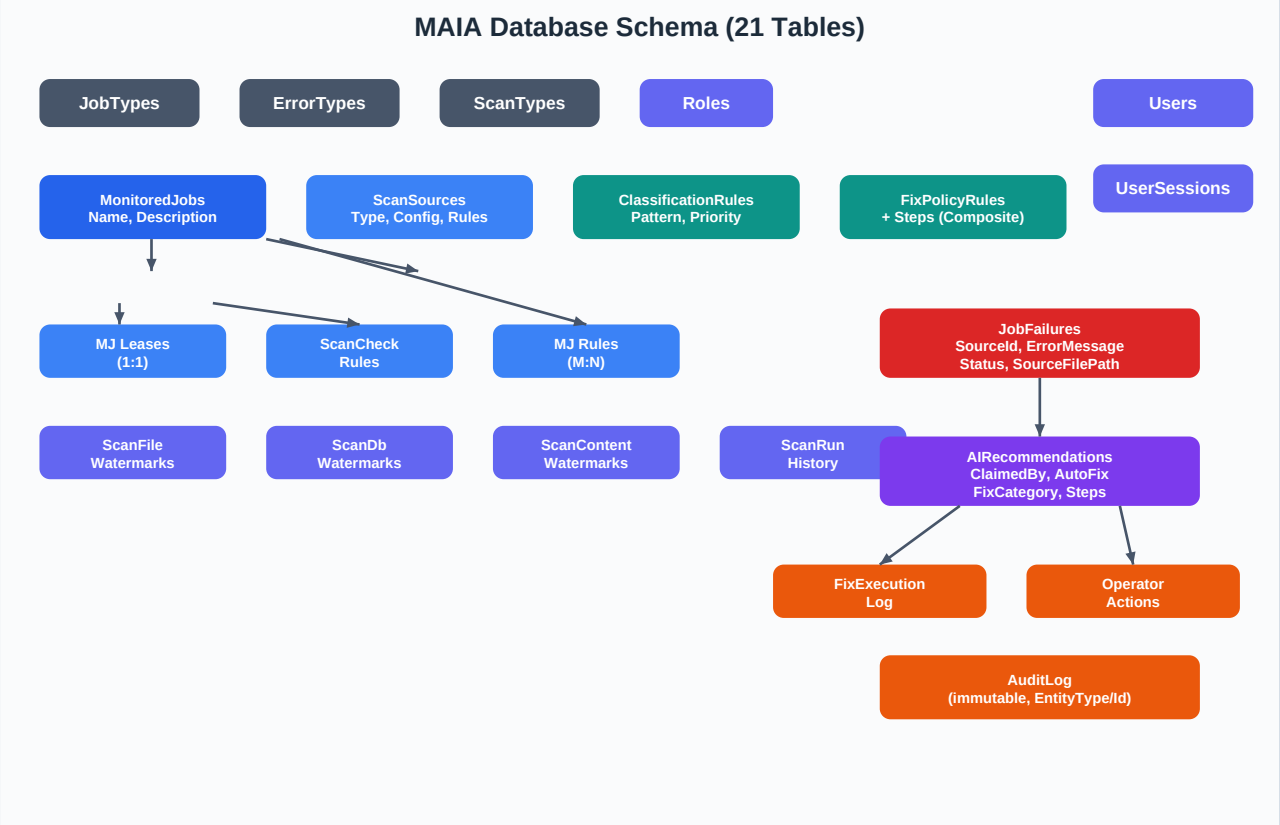


Figure F1 — Database Schema

Table	Description
MonitoredJobs	Identity container: name, description, IsActive. ScanType removed (lives on ScanSource).
ScanSources	Tier 2.5: typed observation point per job. ScanType + config fields + nested rules.
ScanCheckRules	Per-source conditions: ColumnRange, ValueEquals, ErrorKeyword, FileContent. FK → ScanSourceId.
MonitoredJobRules	M:N: MonitoredJob ↔ ClassificationRule
MonitoredJobLeases	1:1 with MonitoredJob. Runtime lease state. ON DELETE CASCADE.
ScanTypes	Lookup: FileSystem, Database, ApiEndpoint, FileContent + LeaseDurationSeconds
ScanDbWatermarks	ISO datetime2 watermark per (ScanSourceId, RuleId)
ScanFileWatermarks	Byte-offset watermark per (ScanSourceId, FilePath)
ScanContentWatermarks	Mtime-based watermark per (ScanSourceId, FilePath)
JobTypes	Lookup: DTSX, EXE, Service, …
ErrorTypes	Soft-delete. Code unique.
ClassificationRules	Wildcard pattern → (JobTypeId, ErrorTypeId). Priority + Confidence.
JobFailures	Detected failure. Status: Failed / Resolved / ManualRequired / AwaitingManualAction. Has SourceFilePath.
AIRecommendations	Fix suggestion. ClaimedBy/ClaimedAt for atomic claim. AutoFixAvailable snapshot.
FixPolicyRules	(JobTypeId, ErrorTypeId) + optional MonitoredJobId. Unique per layer (filtered indexes).

FixPolicyRuleSteps	Ordered steps for Composite policies. FK ON DELETE CASCADE.
FixExecutionLog	Per-attempt result. ResultDetail carries executor failure reason.
OperatorActions	Approve/Reject/Retry records.
AuditLog	Immutable. EntityType + EntityId + nullable FailureId. EventType convention: {Entity}{Verb}.
Roles	Seed: User(1), Operator(2), Administrator(3)
Users	Username (unique CI), PBKDF2 PasswordHash, RoleId FK, MustChangePassword
UserSessions	Token, UserId FK cascade, LastActivityAt. Sliding idle expiry.
ScanRunHistory	Append-only per-tick. INT IDENTITY PK. Pruned after 30 days.

G Appendix — API Endpoint Catalog

Auth (AllowAnonymous: login, me, health)

Method	Endpoint	Description
POST	/api/auth/login	Login → Set-Cookie maia_session
POST	/api/auth/logout	Logout + delete session
POST	/api/auth/change-password	Change password (forced rotation)
GET	/api/auth/me	Current user info [AllowAnonymous]

Data (User tier — read-only)

Method	Endpoint	Description
GET	/api/data/jobs?page=&pageSize=&sort=&dir=	Paginated failures (server-side sort)
GET	/api/data/failures/{id}/status	Failure detail + recs + execution history
GET	/api/data/recommendations?page=&pageSize=	Paginated recommendations + live policy
GET	/api/data/operator-actions?page=&pageSize=	Operator action history (Approve/Reject/Retry)
GET	/api/data/monitored-jobs	All active monitored jobs
GET	/api/data/scan-runs?…	Scan run history (paged, max 200)
GET	/api/data/worker-status	Worker heartbeat for dashboard polling
GET	/api/data/dashboard-stats	KPI stats incl. fixFailedToday
GET	/api/data/analytics/failures-over-time	Time-bucketed failure counts by ErrorType
GET	/api/data/analytics/failures-by-job	Top N jobs by failure count

Operator Actions (Operator tier)

Method	Endpoint	Description
POST	/api/recommendations/{id}/approve	Approve + synchronous drain
POST	/api/recommendations/{id}/reject	Reject (last-pending → ManualRequired)
POST	/api/recommendations/{id}/retry	Retry failed execution with current policy
POST	/api/failures/{id}/mark-resolved	Mark AwaitingManualAction/ManualRequired → Resolved
POST	/api/jobscan/{id}	Trigger scan for one job
POST	/api/jobscan/scan-all	Scan all active jobs
POST	/api/fix/execute-fixes	Manual global drain

Config (Operator read / Admin write)

Method	Endpoint	Description
CRUD	/api/config/monitored-jobs[/{id}]	MonitoredJob CRUD + GET single job with sources
CRUD	/api/config/scan-sources[/{id}]	ScanSource CRUD per job
CRUD	/api/config/scan-rules[/{id}]	ScanCheckRule CRUD per source
CRUD	/api/config/classification-rules[/{id}]	Global ClassificationRule CRUD
CRUD	/api/config/fix-policy-rules[/{id}]	FixPolicyRule CRUD (with steps for composite)

CRUD	/api/config/error-types[/{id}]	ErrorType CRUD (soft-delete)

Admin (Administrator tier)

Method	Endpoint	Description
GET	/api/admin/audit-log	Paged/filtered audit log viewer
POST	/api/admin/scan-history/cleanup	On-demand retention sweep
POST	/api/admin/worker/pause|resume	Pause/resume MonitoringWorker
CRUD	/api/admin/users[/{id}]	User management (create, update, reset-password)

H Appendix — Non-Functional Requirements

Concern	Approach
Reliability	Worker restarts on crash. Lease expiry + recommendation claim timeout allow recovery. Startup drain.
Offline / Air-Gap	No cloud dependency at runtime. All rules in SQL Server. Auth is MAIA-local (no AD in v1).
Horizontal Scale	Multiple workers via MonitoredJobLeases + recommendation atomic claim (READPAST).
Auditability	Immutable AuditLog with EntityType/EntityId. Config changes, fix executions, operator actions all logged.
Security	PBKDF2-HMAC-SHA512, httpOnly/SameSite=Strict cookies, forced rotation, default-CLOSED auth, auth matrix tests.
Extensibility	New scan type → IScanStrategy. New executor → IFixActionExecutor. LLM → IClassificationStrategy.
Incremental Scanning	3 watermark types: byte offset (FS), ISO datetime2 (DB), mtime (FileContent). DB advances within filtered rows only.
Retention	ScanRunHistory pruned after 30 days. Serilog file logs: daily rolling + 30-day retention.

I Appendix — Known Follow-ups

Item	Detail
AD authentication (v2)	MAIA-local auth is v1. Active Directory integration for SSO is planned but not built.
Dashboard Phase 2	Failures by Job horizontal bar + Resolution Mix stacked bar. Backend endpoints ready.
Full RTL/Hebrew	Language switcher scaffold built. Full translations + layout mirroring deferred.
DbFixHandler stub	Returns false → ManualRequired. Either implement or emit clearer diagnostic.
ILogReader removal	No longer consumed since classifier uses JobFailure.ErrorMessage. Safe to remove.
Guided config wizard	Operator wants an assisted job→source→rules creation flow. Design discussion pending.
FixExecutionLog retention	No retention worker yet — table grows unbounded. Add a sweep similar to ScanRunHistory.